

zlib — Compression compatible with gzip

For applications that require data compression, the functions in this module allow compression and de-compression, using the [zlib library](#).

This is an [optional module](#). If it is missing from your copy of CPython, look for documentation from your distributor (that is, whoever provided Python to you). If you are the distributor, see [Requirements for optional modules](#).

zlib's functions have many options and often need to be used in a particular order. This documentation doesn't attempt to cover all of the permutations; consult the [zlib manual](#) for authoritative information.

For reading and writing .gz files see the [gzip](#) module.

The available exception and functions in this module are:

exception **zlib.error**

Exception raised on compression and decompression errors.

zlib.adler32(*data* [, *value*])

Computes an Adler-32 checksum of *data*. (An Adler-32 checksum is almost as reliable as a CRC32 but can be computed much more quickly.) The result is an unsigned 32-bit integer. If *value* is present, it is used as the starting value of the checksum; otherwise, a default value of 1 is used. Passing in *value* allows computing a running checksum over the concatenation of several inputs. The algorithm is not cryptographically strong, and should not be used for authentication or digital signatures. Since the algorithm is designed for use as a checksum algorithm, it is not suitable for use as a general hash algorithm.

Changed in version 3.0: The result is always unsigned.

zlib.compress(*data*, *level*, *level*=Z_DEFAULT_COMPRESSION, *wbits*=MAX_WBITS)

Compresses the bytes in *data*, returning a bytes object containing compressed data. *level* is an integer from 0 to 9 or -1 controlling the level of compression; See [Z_BEST_SPEED](#) (1), [Z_BEST_COMPRESSION](#) (9), [Z_NO_COMPRESSION](#) (0), and the default, [Z_DEFAULT_COMPRESSION](#) (-1) for more information about these values.

The *wbits* argument controls the size of the history buffer (or the "window size") used when compressing data, and whether a header and trailer is included in the output. It can take several ranges of values, defaulting to 15 ([MAX_WBITS](#)):

- +9 to +15: The base-two logarithm of the window size, which therefore ranges between 512 and 32768. Larger values produce better compression at the expense of greater memory usage. The resulting output will include a zlib-specific header and trailer.
- -9 to -15: Uses the absolute value of *wbits* as the window size logarithm, while producing a raw output stream with no header or trailing checksum.
- +25 to +31 = 16 + (9 to 15): Uses the low 4 bits of the value as the window size logarithm, while

including a basic **gzip** header and trailing checksum in the output.

Raises the [error](#) exception if any error occurs.

Changed in version 3.6: *level* can now be used as a keyword parameter.

Changed in version 3.11: The *wbits* parameter is now available to set window bits and compression type.

```
zlib.compressobj(level=Z_DEFAULT_COMPRESSION, method=DEFLATED,
wbits=MAX_WBITS, memLevel=DEF_MEM_LEVEL, strategy=Z_DEFAULT_STRATEGY[,
zdict])
```

Returns a compression object, to be used for compressing data streams that won't fit into memory at once.

level is the compression level – an integer from 0 to 9 or -1. See [Z_BEST_SPEED](#) (1), [Z_BEST_COMPRESSION](#) (9), [Z_NO_COMPRESSION](#) (0), and the default, [Z_DEFAULT_COMPRESSION](#) (-1) for more information about these values.

method is the compression algorithm. Currently, the only supported value is [DEFLATED](#).

The *wbits* parameter controls the size of the history buffer (or the “window size”), and what header and trailer format will be used. It has the same meaning as [described for compress\(\)](#).

The *memLevel* argument controls the amount of memory used for the internal compression state. Valid values range from 1 to 9. Higher values use more memory, but are faster and produce smaller output.

strategy is used to tune the compression algorithm. Possible values are [Z_DEFAULT_STRATEGY](#), [Z_FILTERED](#), [Z_HUFFMAN_ONLY](#), [Z_RLE](#) and [Z_FIXED](#).

zdict is a predefined compression dictionary. This is a sequence of bytes (such as a [bytes](#) object) containing subsequences that are expected to occur frequently in the data that is to be compressed. Those subsequences that are expected to be most common should come at the end of the dictionary.

Changed in version 3.3: Added the *zdict* parameter and keyword argument support.

```
zlib.crc32(data[, value])
```

Computes a CRC (Cyclic Redundancy Check) checksum of *data*. The result is an unsigned 32-bit integer. If *value* is present, it is used as the starting value of the checksum; otherwise, a default value of 0 is used. Passing in *value* allows computing a running checksum over the concatenation of several inputs. The algorithm is not cryptographically strong, and should not be used for authentication or digital signatures. Since the algorithm is designed for use as a checksum algorithm, it is not suitable for use as a general hash algorithm.

Changed in version 3.0: The result is always unsigned.

```
zlib.decompress(data, /, wbits=MAX_WBITS, bufsize=DEF_BUF_SIZE)
```

Decompresses the bytes in *data*, returning a bytes object containing the uncompressed data. The

wbits parameter depends on the format of *data*, and is discussed further below. If *bufsize* is given, it is used as the initial size of the output buffer. Raises the [error](#) exception if any error occurs.

The *wbits* parameter controls the size of the history buffer (or “window size”), and what header and trailer format is expected. It is similar to the parameter for [compressobj\(\)](#), but accepts more ranges of values:

- +8 to +15: The base-two logarithm of the window size. The input must include a zlib header and trailer.
- 0: Automatically determine the window size from the zlib header. Only supported since zlib 1.2.3.5.
- -8 to -15: Uses the absolute value of *wbits* as the window size logarithm. The input must be a raw stream with no header or trailer.
- +24 to +31 = 16 + (8 to 15): Uses the low 4 bits of the value as the window size logarithm. The input must include a gzip header and trailer.
- +40 to +47 = 32 + (8 to 15): Uses the low 4 bits of the value as the window size logarithm, and automatically accepts either the zlib or gzip format.

When decompressing a stream, the window size must not be smaller than the size originally used to compress the stream; using a too-small value may result in an [error](#) exception. The default *wbits* value corresponds to the largest window size and requires a zlib header and trailer to be included.

bufsize is the initial size of the buffer used to hold decompressed data. If more space is required, the buffer size will be increased as needed, so you don’t have to get this value exactly right; tuning it will only save a few calls to `malloc()`.

Changed in version 3.6: *wbits* and *bufsize* can be used as keyword arguments.

`zlib.decompressobj(wbits=MAX_WBITS[, zdict])`

Returns a decompression object, to be used for decompressing data streams that won’t fit into memory at once.

The *wbits* parameter controls the size of the history buffer (or the “window size”), and what header and trailer format is expected. It has the same meaning as [described for decompress\(\)](#).

The *zdict* parameter specifies a predefined compression dictionary. If provided, this must be the same dictionary as was used by the compressor that produced the data that is to be decompressed.

Note: If *zdict* is a mutable object (such as a [bytearray](#)), you must not modify its contents between the call to `decompressobj()` and the first call to the decompressor’s `decompress()` method.

Changed in version 3.3: Added the *zdict* parameter.

Compression objects support the following methods:

`Compress.compress(data)`

Compress *data*, returning a bytes object containing compressed data for at least part of the data in *data*. This data should be concatenated to the output produced by any preceding calls to the `compress()` method. Some input may be kept in internal buffers for later processing.

`Compress.flush([mode])`

All pending input is processed, and a bytes object containing the remaining compressed output is returned. *mode* can be selected from the constants [Z_NO_FLUSH](#), [Z_PARTIAL_FLUSH](#), [Z_SYNC_FLUSH](#), [Z_FULL_FLUSH](#), [Z_BLOCK](#), or [Z_FINISH](#), defaulting to [Z_FINISH](#). Except [Z_FINISH](#), all constants allow compressing further bytestrings of data, while [Z_FINISH](#) finishes the compressed stream and prevents compressing any more data. After calling `flush()` with *mode* set to [Z_FINISH](#), the [compress\(\)](#) method cannot be called again; the only realistic action is to delete the object.

`Compress.copy()`

Returns a copy of the compression object. This can be used to efficiently compress a set of data that share a common initial prefix.

Changed in version 3.8: Added [copy.copy\(\)](#) and [copy.deepcopy\(\)](#) support to compression objects.

Decompression objects support the following methods and attributes:

`Decompress.unused_data`

A bytes object which contains any bytes past the end of the compressed data. That is, this remains `b""` until the last byte that contains compression data is available. If the whole bytestring turned out to contain compressed data, this is `b""`, an empty bytes object.

`Decompress.unconsumed_tail`

A bytes object that contains any data that was not consumed by the last [decompress\(\)](#) call because it exceeded the limit for the uncompressed data buffer. This data has not yet been seen by the zlib machinery, so you must feed it (possibly with further data concatenated to it) back to a subsequent `decompress()` method call in order to get correct output.

`Decompress.eof`

A boolean indicating whether the end of the compressed data stream has been reached.

This makes it possible to distinguish between a properly formed compressed stream, and an incomplete or truncated one.

Added in version 3.3.

`Decompress.decompress(data, max_length=0)`

Decompress *data*, returning a bytes object containing the uncompressed data corresponding to at least part of the data in *string*. This data should be concatenated to the output produced by any preceding calls to the `decompress()` method. Some of the input data may be preserved in internal buffers for later processing.

If the optional parameter *max_length* is non-zero then the return value will be no longer than *max_length*. This may mean that not all of the compressed input can be processed; and uncon-

sumed data will be stored in the attribute [unconsumed_tail](#). This bytestring must be passed to a subsequent call to `decompress()` if decompression is to continue. If `max_length` is zero then the whole input is decompressed, and `unconsumed_tail` is empty.

Changed in version 3.6: `max_length` can be used as a keyword argument.

`Decompress.flush([length])`

All pending input is processed, and a bytes object containing the remaining uncompressed output is returned. After calling `flush()`, the [decompress\(\)](#) method cannot be called again; the only realistic action is to delete the object.

The optional parameter `length` sets the initial size of the output buffer.

`Decompress.copy()`

Returns a copy of the decompression object. This can be used to save the state of the decompressor midway through the data stream in order to speed up random seeks into the stream at a future point.

Changed in version 3.8: Added [copy.copy\(\)](#) and [copy.deepcopy\(\)](#) support to decompression objects.

The following constants are available to configure compression and decompression behavior:

`zlib.DEFLATED`

The deflate compression method.

`zlib.MAX_WBITS`

The maximum window size, expressed as a power of 2. For example, if `MAX_WBITS` is 15 it results in a window size of 32 KiB.

`zlib.DEF_MEM_LEVEL`

The default memory level for compression objects.

`zlib.DEF_BUF_SIZE`

The default buffer size for decompression operations.

`zlib.Z_NO_COMPRESSION`

Compression level 0; no compression.

Added in version 3.6.

`zlib.Z_BEST_SPEED`

Compression level 1; fastest and produces the least compression.

`zlib.Z_BEST_COMPRESSION`

Compression level 9; slowest and produces the most compression.

`zlib.Z_DEFAULT_COMPRESSION`

Default compression level (-1); a compromise between speed and compression. Currently equivalent to compression level 6.

zlib.Z_DEFAULT_STRATEGY

Default compression strategy, for normal data.

zlib.Z_FILTERED

Compression strategy for data produced by a filter (or predictor).

zlib.Z_HUFFMAN_ONLY

Compression strategy that forces Huffman coding only.

zlib.Z_RLE

Compression strategy that limits match distances to one (run-length encoding).

This constant is only available if Python was compiled with zlib 1.2.0.1 or greater.

| *Added in version 3.6.*

zlib.Z_FIXED

Compression strategy that prevents the use of dynamic Huffman codes.

This constant is only available if Python was compiled with zlib 1.2.2.2 or greater.

| *Added in version 3.6.*

zlib.Z_NO_FLUSH

Flush mode 0. No special flushing behavior.

| *Added in version 3.6.*

zlib.Z_PARTIAL_FLUSH

Flush mode 1. Flush as much output as possible.

zlib.Z_SYNC_FLUSH

Flush mode 2. All output is flushed and the output is aligned to a byte boundary.

zlib.Z_FULL_FLUSH

Flush mode 3. All output is flushed and the compression state is reset.

zlib.Z_FINISH

Flush mode 4. All pending input is processed, no more input is expected.

zlib.Z_BLOCK

Flush mode 5. A deflate block is completed and emitted.

This constant is only available if Python was compiled with zlib 1.2.2.2 or greater.

| *Added in version 3.6.*

zlib.Z_TREES

Flush mode 6, for inflate operations. Instructs inflate to return when it gets to the next deflate block boundary.

This constant is only available if Python was compiled with zlib 1.2.3.4 or greater.

Added in version 3.6.

Information about the version of the zlib library in use is available through the following constants:

`zlib.ZLIB_VERSION`

The version string of the zlib library that was used for building the module. This may be different from the zlib library actually used at runtime, which is available as [ZLIB_RUNTIME_VERSION](#).

`zlib.ZLIB_RUNTIME_VERSION`

The version string of the zlib library actually loaded by the interpreter.

Added in version 3.3.

`zlib.ZLIBNG_VERSION`

The version string of the zlib-ng library that was used for building the module if zlib-ng was used. When present, the [ZLIB_VERSION](#) and [ZLIB_RUNTIME_VERSION](#) constants reflect the version of the zlib API provided by zlib-ng.

If zlib-ng was not used to build the module, this constant will be absent.

Added in version 3.14.

See also:

Module [gzip](#)

Reading and writing **gzip**-format files.

<https://www.zlib.net>

The zlib library home page.

<https://www.zlib.net/manual.html>

The zlib manual explains the semantics and usage of the library's many functions.

In case gzip (de)compression is a bottleneck, the [python-isal](#) package speeds up (de)compression with a mostly compatible API.